

MEDIATEK

MT8668 Yocto T-Box User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	吴亚丽	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 T-Box 5

1.1 概述 5

1.1.1 简单介绍 5

1.1.2 T-Box 缩略词 5

1.2 架构/进程概述 6

1.2.1 T-Box 架构 6

1.2.2 T-Box API 说明 6

1.3 常见问题/故障排除 36

1.3.1 SIM/CALL/SMS/Telephony 驻网相关 36

1.3.2 Data 相关 37

1.3.3 网络相关 37

1.3.4 VOLTE 和 VONR 相关 37

附件一 附加条款 38

图片目录

图 1-1. T-Box 架构 6

表格目录

表 1-1. 缩略词 5

表 1-2. Modem 状态及 IMEI 接口说明 6

表 1-3. SIM 接口说明 7

表 1-4. Telephony 网路接口说明 11

表 1-5. 数据接口说明 16

表 1-6. 电话接口说明 24

表 1-7. 短信接口说明 28

表 1-8. IMS 接口说明29

表 1-9. Ecall control 接口说明30

表 1-10. AT 黑名单接口说明35

表 1-11. AT command 接口说明35

表 1-12. MIPC keep alive 接口说明36

1 T-Box

1.1 概述

1.1.1 简单介绍

本文介绍 Hypervisor T-Box 架构、API 使用及注意事项。

1.1.2 T-Box 缩略词

表 1-1.缩略词

缩略词	全称	释义
API	Application Programming Interface	应用程序编程接口
APN	Access Point Name	接入点
T-Box	Telematics-BOX	车联网控制单元

1.2 架构/进程概述

1.2.1 T-Box 架构

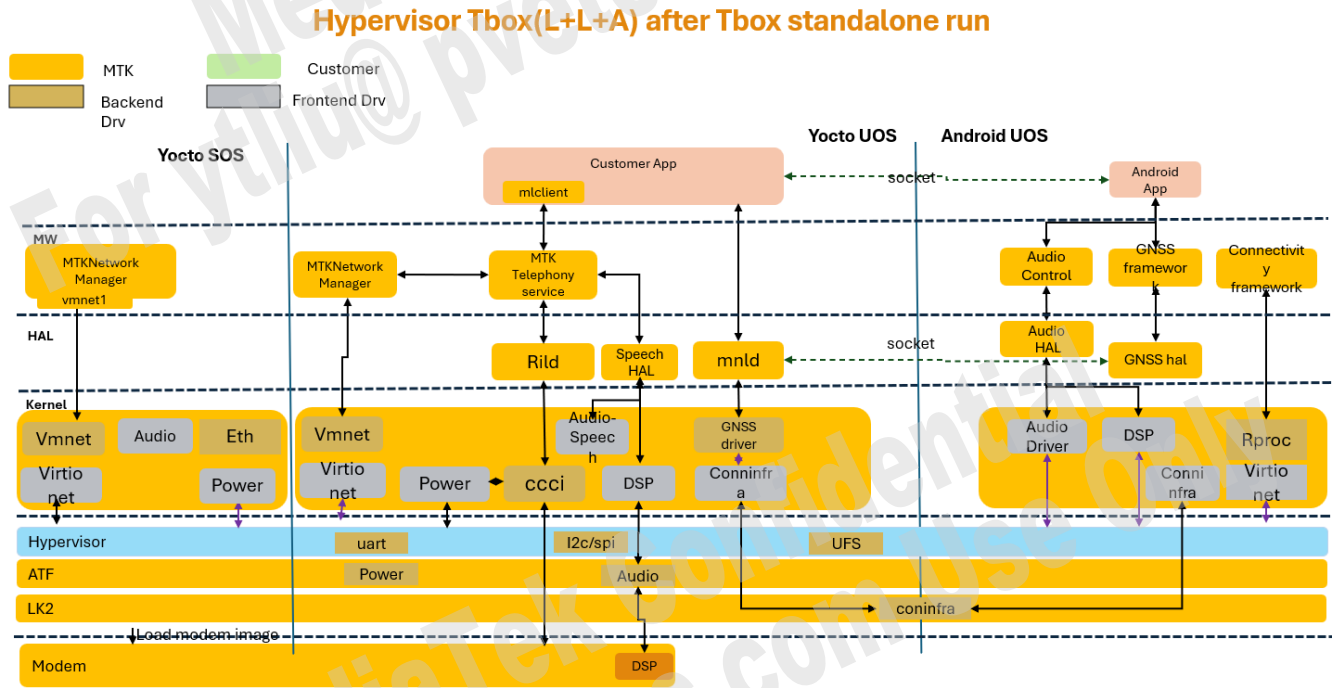


图 1-1. T-Box 架构

1.2.2 T-Box API 说明

1.2.2.1 Modem 状态/IMEI 接口及调用时序说明

T-Box API 除 ML_GetModemStat，其他均需要基于 modem ready 的情况下才能正常工作。

表 1-2. Modem 状态及 IMEI 接口说明

接口/结构体	描述
Int ML_GetModemStat(char *stat, uint32_t statlen)	获取 modem 状态，stat 返回字符串“ready”表示 modem 已正常启动成功
Int ML_GetImei(char* imei, size_t imeiLen)	获取平台 IMEI int 返回值。成功时返回 0; 错误时返回 -1
int ML_GetImeiWithSlot(int32_t slotId, char* imei, size_t imeiLen)	By Slot 获取平台 IMEI

1.2.2.2 SIM 接口及调用时序说明

SIM 接口除 ML_Sim_GetCardStatus，其他均需要基于 SIM present 的情况下才能正常获取到值。

表 1-3. SIM 接口说明

接口/结构体	描述
<pre>typedef enum { E_ML_SIM_CARD_STATE_UNKNOWN = 0xB01, /**< Card state unknown. */ E_ML_SIM_CARD_STATE_ABSENT = 0xB02, /**< Card is absent. */ E_ML_SIM_CARD_STATE_PRESENT = 0xB03, /**< Card is present. */ E_ML_SIM_CARD_STATE_ERROR_UNKNOWN = 0xB04, /**< Unknown error state. */ E_ML_SIM_CARD_STATE_ERROR_POWER_DOWN = 0xB05, /**< Power down. */ E_ML_SIM_CARD_STATE_ERROR_POLL_ERROR = 0xB06, /**< Poll error. */ E_ML_SIM_CARD_STATE_ERROR_NO_ATR_RECEIVED = 0xB07, /**< Failed to receive an answer to reset. */ E_ML_SIM_CARD_STATE_ERROR_VOLT_MISMATCH = 0xB08, /**< Voltage mismatch. */ E_ML_SIM_CARD_STATE_ERROR_PARITY_ERROR = 0xB09, /**< Parity error. */ E_ML_SIM_CARD_STATE_ERROR_SIM_TECHNICAL_PROBLEMS= 0xB0A, /**< Card returned technical problems. */ }E_ML_SIM_CARD_STATE_TYPE_T; /**< Card state. */</pre>	SIM 卡状态结构体。主要关注 PRESENT 为卡已识别，ABSENT 为没有识别到 SIM 卡。
<pre>Int32_t ML_Sim_GetCardStatus(ML_SIM_CARD_STATUS_INFO_T *pvsSimStatus)</pre>	获取 SIM 状态。pvsSimStatus 返回 “present” 表示 SIM 卡在位。 int 返回值。成功时返回 0; 错误时返回 -1
<pre>Int ML_SIM_GetICCID(char *iccid, size_t iccidLen)</pre>	获取 SIM 卡 ICCID。ICCID (Integrated Circuit Card Identifier, 集成电路卡识别码) 是用于唯一标识每张 SIM 卡的编号。 int 返回值。成功时返回 0; 错误时返回 -1
<pre>int ML_SIM_GetICCIDWithSlot(int32_t slotId, char* iccid, size_t iccidLen);</pre>	By Slot 获取 SIM 卡 ICCID
<pre>Int ML_SIM_GetMsisdn(char *msisdn, size_t msisdnLen)</pre>	获取 SIM 卡 Msisdn。这里指电话号码。 int 返回值。成功时返回 0; 错误时返回 -1
<pre>Int ML_SIM_GetImsi(char *imsi, size_t imsilen)</pre>	获取 SIM 卡 IMSI。IMSI (International Mobile

接口/结构体	描述
	Subscriber Identity, (国际移动用户识别码) 是用于唯一标识移动网络中的用户的编号。 int 返回值。成功时返回 0; 错误时返回 -1
int ML_SIM_GetImsiWithSlot(int32_t slotId, char* imsi, size_t imsiLen);	By Slot 获取 SIM 卡 IMSI
int ML_SIM_GetMccMnc(char *mcc, size_t mccLen, char*mnc, size_t mncLen)	获取 SIM 卡 mccmnc。MCCMNC (Mobile Country Code and Mobile Network Code, 移动国家代码和移动网络代码) 是用于唯一标识移动网络运营商的编号。MCCMNC 由两个部分组成: MCC (移动国家代码) 和 MNC (移动网络代码)。 int 返回值。成功时返回 0; 错误时返回 -1
int ML_SIM_GetMccMncWithSlot(int32_t slotId, char* mcc, size_t mccLen, char* mnc, size_t mncLen);	By Slot 获取 SIM 卡 mccmnc
int32_t ML_Sim_GetCardFullStatus (ML_SIM_CARD_FULL_STATUS_INFO_T * pvsSimStatus)	获取完整的 SIM 卡信息。 返回的结构体中包含卡状态、pin 状态、SIM app 信息等。 int 返回值。成功时返回 0; 错误时返回 -1
int32_t ML_Sim_GetCardFullStatusWithSlot(int32_t slotId, ML_SIM_CARD_FULL_STATUS_INFO_T *pvsSimStatus);	By Slot 获取完整的 SIM 卡信息。
typedef struct { E_ML_CardState card_state; E_ML_PinState universal_pin_state; /* applicable to USIM and CSIM: E_ML_PINSTATE_xxx */ int gsm_umts_subscription_app_index; /* value < E_ML_CARD_MAX_APPS, -1 if none */ int cdma_subscription_app_index; /* value < E_ML_CARD_MAX_APPS, -1 if none */ int ims_subscription_app_index; /* value < E_ML_CARD_MAX_APPS, -1 if none */ int num_applications; /* value <= E_ML_CARD_MAX_APPS E_ML_AppStatus applications[E_ML_CARD_MAX_APPS]; } ML_SIM_CARD_FULL_STATUS_INFO_T;	SIM 卡 详细信息结构体。
int ML_SIM_ChangePin (int32_t slotIndex, const char * oldPin, const char * newPin, const char * aid, int32_t * errorCode, int32_t * retryTimes)	PIN1: 用于保护 SIM 卡, 防止未经授权的使用。 PIN2: 用于保护特定的 SIM 卡功能, 如固定拨号号码。 PUK: 用于解锁被 PIN1 锁定的 SIM 卡。

接口/结构体	描述
	PUK2：用于解锁被 PIN2 锁定的功能。 修改 pin 值。 注意：需要上锁之后才能 change pin int 返回值 On success, 0 is returned. On error, -1 is returned. 出参：errorCode; On error, errorCode is returned。errorCode 可参考 ril.h 中的 RIL_Errno 枚举。 出参：retryTimes 可尝试次数、默认为 3，pin 输入错误时返回-1。
<pre>int ML_SIM_ChangePin2 (int32_t slotIndex, const char * oldPin2, const char * newPin2, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	修改 pin2 值。 注意：需要上锁之后才能 change pin2 int 返回值。成功时返回 0; 错误时返回 -1 出参：errorCode; On error, errorCode is returned。errorCode 可参考 ril.h 中的 RIL_Errno 枚举。 出参：retryTimes 可尝试次数、默认为 3，pin 输入错误时返回-1。
<pre>int ML_SIM_EnterPin (int32_t slotIndex, const char * pin, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	解锁 pin。 注意：需要上锁之后才能解锁 pin int 返回值。成功时返回 0; 错误时返回 -1 出参：errorCode; On error, errorCode is returned。errorCode 可参考 ril.h 中的 RIL_Errno 枚举。 出参：retryTimes 可尝试次数、默认为 3，pin 输入错误时返回-1。
<pre>int ML_SIM_EnterPin2 (int32_t slotIndex, const char * pin2, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	解锁 pin2。 注意：需要上锁之后才能解锁 pin2 int 返回值。成功时返回 0; 错误时返回 -1 出参：errorCode; On error, errorCode is returned。errorCode 可参考 ril.h 中的 RIL_Errno 枚举。

接口/结构体	描述
	出参： retryTimes 可尝试次数、默认为 3， pin2 输入错误时返回-1.
<pre>int ML_SIM_EnterPuk (int32_t slotIndex, const char * puk, const char * pin, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	解锁 PUK。用于解锁被 PIN1 锁定的 SIM 卡。当用户连续输入错误的 PIN1 超过一定次数后，SIM 卡会被锁定，此时需要输入 PUK 码才能解锁并重置 PIN1。 注意：需要上锁之后才能解锁 puk int 返回值。成功时返回 0；错误时返回 -1 出参： errorCode； On error, errorCode is returned。errorCode 可参考 ril.h 中的 RIL_Errno 枚举。 出参： retryTimes 可尝试次数、默认为 3， puk 输入错误时返回-1.
<pre>int ML_SIM_EnterPuk2 (int32_t slotIndex, const char * puk2, const char * pin2, const char * aid, int32_t * errorCode, int32_t * retryTimes)</pre>	解锁 PUK2。用于解锁被 PIN2 锁定的 SIM 卡。当用户连续输入错误的 PIN2 超过一定次数后，SIM 卡会被锁定，此时需要输入 PUK 码才能解锁并重置 PIN2。 注意：需要上锁之后才能解锁 puk int 返回值。成功时返回 0；错误时返回 -1 出参： errorCode； On error, errorCode is returned。errorCode 可参考 ril.h 中的 RIL_Errno 枚举。 出参： retryTimes 可尝试次数、默认为 3， puk2 输入错误时返回-1.
<pre>int ML_SIM_SetFacilityLock (int32_t slotIndex, const char * facility, int32_t lockState, const char * password, int32_t serviceClass, const char * appld, int32_t * errorCode, int32_t * retryTimes)</pre>	设置 pin 锁状态。 注意：没有上锁的时候才能设置，如果在上锁的情况下调用本函数会返回 error。 入参： facility：设备类型 lockState： 1：上锁 0：解锁 int 返回值。成功时返回 0；错误时返回 -1
<pre>int ML_SIM_IccCloseLogicalChannelBySlot (int32_t slotIndex, int32_t channel)</pre>	关闭 IO 逻辑通道。 int 返回值。成功时返回 0；错误时返回 -1
<pre>int ML_SIM_IccOpenLogicalChannelBySlot (int32_t slotIndex, const char * aid, int32_t p2, int32_t channel)</pre>	打开 IO 逻辑通道。 int 返回值。成功时返回 0；错误时返回 -1

接口/结构体	描述
<pre>int ML_SIM_IccTransmitApuLogicalChannelBySlot (int32_t slotIndex, int32_t channel, int32_t cla, int32_t instruction, int32_t p1, int32_t p2, int32_t p3, const char * data)</pre>	发送 apdu 命令。 入参: int32_t channel: 逻辑通道 int32_t cla: APDU 命令的类字节 (Class byte) int32_t instruction: APDU 命令的指令字节 (Instruction byte) 。 int32_t p1, int32_t p2, int32_t p3, 指 APDU 命令的参数。用于进一步指定命令的操作 const char * data APDU 命令的数据字段 int 返回值。成功时返回 0; 错误时返回 -1
<pre>int ML_SIM_SetCardPower (int32_t CardPowerState)</pre>	设置 SIM 卡 Power 状态, 也就是 SIM 卡复位。 入参: int32_t CardPowerState 0: power off 1: power on 2: 透传模式 int 返回值。成功时返回 0; 错误时返回 -1

1.2.2.3 Telephony 网络接口及调用时序说明

表 1-4. Telephony 网路接口说明

接口/结构体	描述
<pre>typedef enum { E_ML_NW_PS_UNKNOWN = 0x00, E_ML_NW_PS_ATTACHED = 0x01, E_ML_NW_PS_DETACHED = 0x02, }E_ML_NW_PS_REG_STATE_TYPE_T;</pre>	PS 网络注册状态结构体。
<pre>typedef enum { E_ML_NW_CS_UNKNOWN = 0x00, E_ML_NW_CS_ATTACHED = 0x01, E_ML_NW_CS_DETACHED = 0x02, }E_ML_NW_CS_REG_STATE_TYPE_T;</pre>	CS 网络注册状态结构体。
<pre>int ML_GetNetState(E_ML_NW_PS_REG_STATE_TYPE_T * ps, E_ML_NW_CS_REG_STATE_TYPE_T * cs)</pre>	获取 CS/PS 网络注册状态。 int 返回值。成功时返回 0; 错误时返回 -1 注意: 此接口需在识卡后调用
<pre>int ML_GetNetStateWithSlot(int32_t slotId, E_ML_NW_PS_REG_STATE_TYPE_T *ps,</pre>	By slot 获取 CS/PS 网络注册状态。

接口/结构体	描述
E_ML_NW_CS_REG_STATE_TYPE_T *cs)	
int32_t ML_GetOperatorCode (uint8_t * op_code))	获取运营商名称信息. op_code 为 1 表示 SIM 运营商是中国联通, 为 0 表示中国移动。 int 返回值。成功时返回 0; 错误时返回 -1 注意：此接口需在识卡后调用
int32_t ML_GetOperatorCodeWithSlot(int32_t slotId, uint8_t *op_code);	By slot 获取运营商名称信息
typedef enum { E_ML_NW_RAT_GSM = 0, E_ML_NW_RAT_UTRAN = 2, E_ML_NW_RAT_GSMW_EGPRS = 3, E_ML_NW_RAT_UTRANW_HSDPA = 4, E_ML_NW_RAT_UTRANW_HSUPA = 5, E_ML_NW_RAT_UTRANW_HSDPA_AND_HSUPA = 6, E_ML_NW_RAT_E_UTRAN = 7, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_NW_RAT_NR = 8, #endif }E_ML_NW_RADIO_ACCESS_TYPE_T;	网络制式类型结构体。
int ML_GetRadioAccessType (E_ML_NW_RADIO_ACCESS_TYPE_T * access_type))	获取驻上网络的制式类型。 int 返回值。成功时返回 0; 错误时返回 -1 注意：此接口需在识卡后调用
int ML_GetRadioAccessTypeWithSlot(int32_t slotId, E_ML_NW_RADIO_ACCESS_TYPE_T *access_type))	By slot 获取驻上网络的制式类型。
typedef struct { ML_GW_SignalStrength GW_SignalStrength; ML_CDMA_SignalStrength CDMA_SignalStrength; ML_EVDO_SignalStrength EVDO_SignalStrength; ML_LTE_SignalStrength LTE_SignalStrength; ML_TD_SCDMA_SignalStrength TD_SCDMA_SignalStrength; ML_WCDMA_SignalStrength WCDMA_SignalStrength; #ifdef SPM_TELEPHONY_NR_SUPPORT ML_NR_SignalStrength NR_SignalStrength; #endif }ML_SignalStrength;	信号强度参数结构体
int ML_GetSignalStrength (int8_t * sig_level))	获取信号强度等级。 sig_level: 0~4, 信号等级阈值参考 AOSP default 划分。

接口/结构体	描述
	int 返回值。成功时返回 0; 错误时返回 -1 注意：此接口需在识卡后调用
int ML_GetSignalStrengthWithSlot(int32_t slotId, int8_t *sig_level)	By Slot 获取信号强度等级。
int ML_SetAirplaneMode (uint8_t on_off)	设置开关飞行模式。on_off 为 1 表示开飞行模式， on_off 为 0 表示关飞行模式。 int 返回值。成功时返回 0; 错误时返回 -1 注意：此接口需在识卡后调用
typedef void (*ml_signal_strength_cb_t)(ML_SignalStrength *state);	信号强度回调函数。
int ML_SignalStreghthInit (ml_signal_strength_cb_t evt_cb)	注册信号强度回调。 int 返回值。成功时返回 0; 错误时返回 -1
typedef enum { E_ML_AUTO = 0, E_ML_2GONLY = 1, E_ML_3GONLY = 2, E_ML_4GONLY = 3, #ifdef SPM_TELEPHONY_NR_SUPPORT E_ML_5GONLY = 4, E_ML_5GAUTO = 5, E_ML_5G4GAUTO = 6, #endif }ml_nw_net_mode_e;	网络模式结构体。
int ML_SetNetMode (ml_nw_net_mode_e mode)	设置网络模式。 int 返回值。成功时返回 0; 错误时返回 -1
int ML_SetNetModeWithSlot(int slot_id, ml_nw_net_mode_e mode)	按 slot 设置网络模式 int 返回值。成功时返回 0; 错误时返回 -1
int ML_GetNetMode (ml_nw_net_mode_e * mode)	获取网络模式。 int 返回值。成功时返回 0; 错误时返回 -1
int ML_GetNetModeWithSlot(int slot_id, ml_nw_net_mode_e *mode)	按 slot 获取网络模式 int 返回值。成功时返回 0; 错误时返回 -1
int ML_GetFullSignalStrength (int slot_id, ML_SignalStrength * fullMsg)	获取所有信号强度信息。 函数返回值为 int 类型。成功时返回 0; 错误时返回-1。

接口/结构体	描述
int getAvailableNetworksWithActAsync (int slot_id)	扫描可用网络 响应函数为 ML_InitAvailableNetworksWithActResponseCb() 函数返回值为 int 类型，成功则返回 0; 错误则返回 -1。
int getCellInfoListAsync (int slot_id)	请求无线电已知的所有当前小区信息。无线电必须返回所有当前小区的列表，包括邻近小区。如果不知道特定小区的信息，则将返回相应的未知值。响应函数是 ML_getCellInfoListResponseCb() 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int ML_getCellInfoListResponseCb (int32_t slot_id, ml_get_cell_info_list_response response)	getCellInfoListAsync 的响应函数 ML_getCellInfoListResponseCb
int32_t getDataRegistrationState (int32_t slot_id)	请求当前数据注册状态，响应函数为 ML_getDataRegistrationStateResponseCb() 函数返回值为 int 类型，成功则返回 0; 错误则返回 -1。
int ML_getDataRegistrationStateResponseCb (int32_t slot_id, ml_get_data_registration_state_response response)	getDataRegistrationState 的响应函数 getDataRegistrationStateResponseCb
int32_t getNetworkSelectionMode (int32_t slot_id, ML_NetworkSelectionMode * mode)	查询当前网络选择模式。 函数返回值为 int 类型，成功则返回 0; 错误则返回 -1。
int32_t getNitzTime (int32_t slot_id, char ** nitzTime, int32_t tz_valid)	获取 NITZ 信息（UTC 时间和时区）。使用 asprintf(&nitzTime, "%02d/%02d/%02d,%02d:%02d:%02d%+03d,%d", pNitzInfo->year % 100, pNitzInfo->month, pNitzInfo->day, pNitzInfo->hour, pNitzInfo->minute, pNitzInfo->second, pNitzInfo->time_zone_offset_minutes / 15, pNitzInfo->daylight_saving_offset_minutes / 60);后需要释放 nitzTime。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int getOperatorSync (int slot_id, char ** longName, char ** shortName, char ** numeric)	请求当前操作员 ONS 或 EONS。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int setUsageSetting (int slot_id, int usageSetting)	设置 UE 使用设置，用于数据/语音中心使用。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int getUsageSetting (int slot_id, int * usageSetting)	获取以数据/语音为中心的 UE 使用设置。

接口/结构体	描述
)	函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t getVoiceRegistrationState (int32_t slot_id)	请求当前语音注册状态, 响应函数为 ML_getVoiceRegistrationStateResponseCb() 函数返回值为 int 类型, 成功则返回 0; 错误则返回-1。
int ML_getVoiceRegistrationStateResponseCb (int32_t slot_id, ml_get_voice_registration_state_response response)	getVoiceRegistrationState 响应函数
int ML_CsNetworkInit (int32_t slot_id, ml_cs_network_change_cb_t evt_cb)	初始化网络模块, 并注册回调函数。 函数返回值为 int 类型, 成功则返回 0; 错误则返回-1。
int ML_PsNetworkInit (int32_t slot_id, ml_ps_network_change_cb_t evt_cb)	初始化 ps 网络模块, 并注册回调函数。 函数返回值为 int 类型, 成功则返回 0; 错误则返回-1。
int ML_imsNetworkStateChangedInit (int32_t slot_id, ml_ims_network_state_changed_cb_t evt_cb)	指示 IMS 注册状态何时发生变化。要获取 IMS 注册状态和 IMS SMS 格式, 被叫方需要调用 getImsRegState()。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。指示语音或数据网络状态何时发生变化。被叫方必须调用 getOperator()、getVoiceRegistrationState()、getDataRegistrationState() 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int ML_networkStateChangedInit (int32_t slot_id, ml_network_state_changed_cb_t evt_cb)	指示语音或数据网络状态何时发生变化。被叫方必须调用 getOperator()、getVoiceRegistrationState()、getDataRegistrationState() 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int ML_nitzTimeReceivedInit (int32_t slot_id, ml_nitz_time_received_t evt_cb)	初始化 nitz 时间变化, 并注册回调函数。 函数返回值为 int 类型。成功时返回 0; 错误时返回-1。
int32_t setNetworkSelectionModeAutomatic (int32_t slot_id)	指定必须自动选择网络。响应函数为 ML_setNetworkSelectionModeAutomaticResponseCb() 函数返回值为 int 类型。成功时返回 0; 错误时返回-1。
int ML_setNetworkSelectionModeAutomaticResponseCb (int32_t slot_id, ml_set_network_selection_mode_automatic_response response)	setNetworkSelectionModeAutomatic 响应函数

接口/结构体	描述
<pre>int32_t setNetworkSelectionModeManual (int32_t slot_id, const char * operatorNumeric, ML_AccessNetwork ran)</pre>	手动选择指定网络。此请求必须在选择并注册新运营商之前不响应。根据 TS 23.122，RAN 只是初始建议值。如果注册失败、之后 RAN 不可用，或者 RAN 不在 IRadioNetwork::setAllowedNetworkTypeBitmap 指定的网络类型内，则调制解调器将需要选择下一个最佳 RAN 进行网络注册。响应函数为 ML_setNetworkSelectionModeManualResponseCb() 函数返回值为 int 类型。成功时返回 0；错误时返回 -1。
<pre>int ML_setNetworkSelectionModeManualResponseCb (int32_t slot_id, ml_set_network_selection_mode_manual_response response)</pre>	setNetworkSelectionModeManual 响应函数
<pre>int ML_SetCellInfoInd (int32_t slot_id, int32_t onoff, ml_cell_info_list_ind evt_cb)</pre>	设置小区信息变化响应函数

1.2.2.4 数据接口及调用时序说明

表 1-5. 数据接口说明

接口/结构体	描述
<pre>typedef enum { ML_DATA_CALL_TYPE_IPV4 = 0, ML_DATA_CALL_TYPE_IPV6, ML_DATA_CALL_TYPE_IPV4V6, } ml_data_call_ip_family_e;</pre>	IP 类型枚举定义。
<pre>typedef enum { ML_APN_PDP_TYPE_IPV4 = 0, ML_APN_PDP_TYPE_PPP, ML_APN_PDP_TYPE_IPV6, ML_APN_PDP_TYPE_IPV4V6, } ml_apn_pdp_type_e;</pre>	APN PDP 类型枚举定义。
<pre>typedef enum { ML_APN_AUTH_PROTO_DEFAULT = 0, ML_APN_AUTH_PROTO_NONE, ML_APN_AUTH_PROTO_PAP, ML_APN_AUTH_PROTO_CHAP, ML_APN_AUTH_PROTO_PAP_CHAP, } ml_apn_auth_proto_e;</pre>	APN 认证协议类型枚举定义。

接口/结构体	描述
<pre>typedef struct { unsigned char profile_idx; ml_apn_pdp_type_e pdp_type; ml_apn_auth_proto_e auth_proto; char apn_name[ML_APN_NAME_SIZE]; char username[ML_APN_USERNAME_SIZE]; char password[ML_APN_PASSWORD_SIZE]; }ml_apn_info_s</pre>	APN 信息结构体 profile_idx: 1 – 8 (1: 公网 ID; 2-8: 私网 ID, 至多支持 7 路私网 APN) pdp_type: IPV4/IPV6/IPV4V6, 详情参考 ml_apn_pdp_type_e enum auth_proto: default/none/pap/chap/pap_chap apn_name: APN 名字
<pre>typedef struct { char apn_type[ML_APN_TYPE_SIZE]; ml_apn_pdp_type_e pdp_type; ml_apn_auth_proto_e auth_proto; char apn_name[ML_APN_NAME_SIZE]; char username[ML_APN_USERNAME_SIZE]; char password[ML_APN_PASSWORD_SIZE]; int bitmask; }ml_apn_info_type_s</pre>	APN 信息结构体 apn_type: default/rcs/bip/ims/... pdp_type: IPV4/IPV6/IPV4V6, 详情参考 ml_apn_pdp_type_e enum auth_proto: default/none/pap/chap/pap_chap apn_name: APN 名字 bitmask: apn bitmask
<pre>typedef enum { ML_DATA_CALL_ERROR_NONE = 0, ML_DATA_CALL_ERROR_INVALID_PARAMS, } ml_data_call_error_e;</pre>	Data 数据连接返回错误码枚举定义。
<pre>typedef enum { ML_DATA_CALL_IDLE, ML_DATA_CALL_CONNECTING, ML_DATA_CALL_CONNECTED, ML_DATA_CALL_DISCONNECTING, ML_DATA_CALL_DISCONNECTED, ML_DATA_CALL_RETRYING, ML_DATA_CALL_FAILED, ML_DATA_CALL_SCANNING, } ml_data_call_state_e;</pre>	Data 数据连接状态枚举定义。
<pre>struct ml_v4_address_status { struct in_addr ip; struct in_addr gateway; struct in_addr pri_dns; struct in_addr sec_dns; };</pre>	IPV4 地址信息结构体。
<pre>struct ml_v6_address_status { struct in6_addr ip; struct in6_addr gateway; struct in6_addr pri_dns; struct in6_addr sec_dns; };</pre>	IPV6 地址信息结构体。

接口/结构体	描述
<pre>struct ml_pkt_stats { unsigned long pkts_tx; unsigned long pkts_rx; long long bytes_tx; long long bytes_rx; unsigned long pkts_dropped_tx; unsigned long pkts_dropped_rx; };</pre>	数据包状态结构体。
<pre>struct ml_v4_info { char name[16]; ml_data_call_state_e state; bool reconnect; struct ml_v4_address_status addr; struct ml_pkt_stats stats; };</pre>	IPV4 信息结构体 name: APN 名字 state: 数据连接状态 reconnect: 重拨标志位 addr: IPV4 地址信息 stats: IPV4 数据包状态
<pre>struct ml_v6_info { char name[16]; ml_data_call_state_e state; bool reconnect; struct ml_v6_address_status addr; struct ml_pkt_stats stats; };</pre>	IPV6 信息结构体 name: APN 名字 state: 数据连接状态 reconnect: 重拨标志位 addr: IPV6 地址信息 stats: IPV6 数据包状态
<pre>typedef struct { char profile_idx; /*!< UMTS/CDMA profile ID. */ ml_data_call_ip_family_e ip_family; /*!< IP version. */ struct ml_v4_info v4; /*!< IPv4 information */ struct ml_v6_info v6; /*!< IPv6 information */ } ml_data_call_info_s;</pre>	Data 数据连接信息结构体。
<pre>typedef struct { char profile_idx; char name[16]; ml_data_call_ip_family_e ip_family; ml_data_call_state_e state; ml_data_call_error_e err; struct ml_v4_address_status v4; struct ml_v6_address_status v6; } ml_data_call_state_s;</pre>	Data 数据连接状态结构体。
<pre>typedef void (*ml_data_call_evt_cb_t)(ml_data_call_state_s *state);</pre>	Data 回调函数定义。 参数： 输入：ml_data_call_state_s *state
<pre>typedef enum { PDP_FAIL_NONE = 0,</pre>	Data 数据连接失败原因枚举定义。

接口/结构体	描述
/* an integer cause code defined in TS 24.008 section 6.1.3.1.3 or TS 24.301 Release 8+ Annex B. If the implementation does not have access to the exact cause codes, then it should return one of the following values, as the UI layer needs to distinguish these cases for error notification and potential retries. */ PDP_FAIL_OPERATOR_BARRED = 0x08, PDP_FAIL_NAS_SIGNALLING = 0x0E, PDP_FAIL_MBMS_CAPABILITIES_INSUFFICIENT = 0x18, PDP_FAIL_LLC_SNDP = 0x19, PDP_FAIL_INSUFFICIENT_RESOURCES = 0x1A, PDP_FAIL_MISSING_UNKNOWN_APN = 0x1B, PDP_FAIL_UNKNOWN_PDP_ADDRESS_TYPE = 0x1C, PDP_FAIL_USER_AUTHENTICATION = 0x1D, PDP_FAIL_ACTIVATION_REJECT_GGSN = 0x1E, PDP_FAIL_ACTIVATION_REJECT_UNSPECIFIED = 0x1F, PDP_FAIL_SERVICE_OPTION_NOT_SUPPORTED = 0x20, PDP_FAIL_SERVICE_OPTION_NOT_SUBSCRIBED = 0x21, PDP_FAIL_SERVICE_OPTION_OUT_OF_ORDER = 0x22, PDP_FAIL_NSAPI_IN_USE = 0x23, PDP_FAIL_REGULAR_DEACTIVATION = 0x24, PDP_FAIL_QOS_NOT_ACCEPTED = 0x25, PDP_FAIL_NETWORK_FAILURE = 0x26, PDP_FAIL_UMTS_REACTIVATION_REQ = 0x27, PDP_FAIL_FEATURE_NOT_SUPP = 0x28, PDP_FAIL_TFT_SEMANTIC_ERROR = 0x29, PDP_FAIL_TFT_SYNTAX_ERROR = 0x2A, PDP_FAIL_UNKNOWN_PDP_CONTEXT = 0x2B, PDP_FAIL_FILTER_SEMANTIC_ERROR = 0x2C, PDP_FAIL_FILTER_SYNTAX_ERROR = 0x2D, PDP_FAIL_PDP_WITHOUT_ACTIVE_TFT = 0x2E, PDP_FAIL_MULTICAST_GROUP_MEMBERSHIP_TIMEOUT = 0x2F, PDP_FAIL_BCM_VIOLATION = 0x30, PDP_FAIL_LAST_PDN_DISC_NOT_ALLOWED = 0x31, PDP_FAIL_ONLY_IPV4_ALLOWED = 0x32, PDP_FAIL_ONLY_IPV6_ALLOWED = 0x33, PDP_FAIL_ONLY_SINGLE_BEARER_ALLOWED = 0x34, PDP_FAIL_ESM_INFO_NOT_RECEIVED = 0x35, PDP_FAIL_PDN_CONN_DOES_NOT_EXIST = 0x36, PDP_FAIL_MULTI_CONN_TO_SAME_PDN_NOT_ALLOWED = 0x37, PDP_FAIL_COLLISION_WITH_NW_INITIATED_REQUEST = 0x38, PDP_FAIL_ESM_UNSUPPORTED_QCI_VALUE = 0x3B, PDP_FAIL_BEARER_HANDLING_NOT_SUPPORT = 0x3C, PDP_FAIL_MAX_ACTIVE_PDP_CONTEXT_REACHED = 0x41, PDP_FAIL_UNSUPPORTED_APN_IN_CURRENT_PLMN = 0x42, PDP_FAIL_INVALID_TRANSACTION_ID = 0x51, PDP_FAIL_MESSAGE_INCORRECT_SEMANTIC = 0x5F, PDP_FAIL_INVALID_MANDATORY_INFO = 0x60, PDP_FAIL_MESSAGE_TYPE_UNSUPPORTED = 0x61, PDP_FAIL_MSG_TYPE_NONCOMPATIBLE_STATE = 0x62, PDP_FAIL_UNKNOWN_INFO_ELEMENT = 0x63, PDP_FAIL_CONDITIONAL_IE_ERROR = 0x64,	

接口/结构体	描述
<pre> PDP_FAIL_MSG_AND_PROTOCOL_STATE_UNCOMPATIBLE = 0x65, PDP_FAIL_PROTOCOL_ERRORS = 0x6F, /* no retry */ PDP_FAIL_APN_TYPE_CONFLICT = 0x70, PDP_FAIL_INVALID_PCSCF_ADDR = 0x71, PDP_FAIL_INTERNAL_CALL_PREEMPT_BY_HIGH_PPIO_APN = 0x72, PDP_FAIL_EMM_ACCESS_BARRED = 0x73, PDP_FAIL_EMERGENCY_IFACE_ONLY = 0x74, PDP_FAIL_IFACE_MISMATCH = 0x75, PDP_FAIL_COMPANION_IFACE_IN_USE = 0x76, PDP_FAIL_IP_ADDRESS_MISMATCH = 0x77, PDP_FAIL_IFACE_AND_POL_FAMILY_MISMATCH = 0x78, PDP_FAIL_EMM_ACCESS_BARRED_INFINITE_RETRY = 0x79, PDP_FAIL_AUTH_FAILURE_ON_EMERGENCY_CALL = 0x7A, PDP_FAIL_LOCAL_REJECT_ACT_REQ_DUE_TO_REACH_RETRY_COUNTER = 0x0EOF, PDP_FAIL_TCM_ESM_TIMER_TIMEOUT = 0x0F46, PDP_FAIL_PAM_ATT_PDN_ACCESS_REJECT_IMS_PDN_BLOCK_TEMP = 0x1402, PDP_FAIL_DATA_NOT_ALLOW = 0x1671, PDP_FAIL_OEM_DCFailCause_1 = 0x1001, PDP_FAIL_OEM_DCFailCause_2 = 0x1002, PDP_FAIL_OEM_DCFailCause_3 = 0x1003, PDP_FAIL_OEM_DCFailCause_4 = 0x1004, PDP_FAIL_OEM_DCFailCause_5 = 0x1005, PDP_FAIL_OEM_DCFailCause_6 = 0x1006, PDP_FAIL_OEM_DCFailCause_7 = 0x1007, PDP_FAIL_OEM_DCFailCause_8 = 0x1008, PDP_FAIL_OEM_DCFailCause_9 = 0x1009, PDP_FAIL_OEM_DCFailCause_10 = 0x100A, PDP_FAIL_OEM_DCFailCause_11 = 0x100B, PDP_FAIL_OEM_DCFailCause_12 = 0x100C, PDP_FAIL_OEM_DCFailCause_13 = 0x100D, PDP_FAIL_OEM_DCFailCause_14 = 0x100E, PDP_FAIL_OEM_DCFailCause_15 = 0x100F, PDP_FAIL_VOICE_REGISTRATION_FAIL = -1, PDP_FAIL_DATA_REGISTRATION_FAIL = -2, PDP_FAIL_SIGNAL_LOST = -3, PDP_FAIL_PREF_RADIO_TECH_CHANGED = -4, PDP_FAIL_RADIO_POWER_OFF = -5, PDP_FAIL_TETHERED_CALL_ACTIVE = -6, PDP_FAIL_ROUTER_ADVERTISEMENT_FAIL = -7, PDP_FAIL_LOST_CONNECTION = 0x10004, PDP_FAIL_ERROR_UNSPECIFIED = 0xffff, }ML_DataCallFailCause; </pre>	
<pre> int ML_APN_Get (unsigned char profile_idx, ml_apn_info_s * apn) </pre>	根据 profile_idx 获取对应的 APN 信息。 参数： 输入： unsigned char profile_idx 输出： ml_apn_info_s *apn

接口/结构体	描述
	APN 初始化流程： 开机识别到卡后, mtktelephonyService 判断/data/vendor/telephony/apn.db 是否存在, 若不存在则根据 /vendor/etc/apns_conf.xml 生成 /data/vendor/telephony/apn.db。再通过卡的 mccmnc 从 apn.db 里获取 SIM 卡的 APN, 设定 APN 到 modem。 注意：此接口需在识卡后调用, 即 ML_Sim_GetCardStatus() 返回 present 后。
<pre>int ML_APN_Set (const ml_apn_info_s * apn)</pre>	设定 APN 到 modem 和 APN database (/data/vendor/telephony/apn.db) 参数： 输入：const ml_apn_info_s * apn 注意：此接口需在识卡后调用
<pre>int ML_APN_Get_ByType (char *apn_type, ml_apn_info_type_s * apn)</pre>	根据 apn_type 获取对应的 APN 信息。 参数： 输入：char *apn_type 输出：ml_apn_info_s *apn
<pre>int ML_APN_Set_ByType (const ml_apn_info_type_s * apn)</pre>	设定 APN 到 modem 和 APN database (/data/vendor/telephony/apn.db) 参数： 输入：const ml_apn_info_type_s * apn 注意：此接口需在识卡后调用
<pre>int ML_DataCallInit (ml_data_call_evt_cb_t evt_cb)</pre>	注册 data call 状态变化 callback 函数, 当 data call 状态发生变化时回调 evt_cb 函数。 参数： 输入：ml_data_call_evt_cb_t evt_cb 注意：限单卡场景使用, 双卡场景下使用 ML_DataCallInitWithSlot。

接口/结构体	描述
<pre>int ML_DataCallInitWithSlot(int32_t slot_id, ml_data_call_evt_cb_t evt_cb)</pre>	注册 data call 状态变化 callback 函数，当 data call 状态发生变化时回调 evt_cb 函数。 参数： 输入： int32_t slot_id 根据 slot_id 注册 datacall 变化状态 输入： ml_data_call_evt_cb_t evt_cb
<pre>int ML_DataCallStart (int32_t profile_idx, ml_data_call_error_e * err)</pre>	建立公网/私网 PDN 连接。 同步接口，下发建立 PDN 命令到 modem，modem 返回后，此接口才返回。profile_idx：1, 公网; 2, 私网(rcs 类型); 3: 私网(bip 类型); 参数： 输入： int32_t profile_idx 输出： ml_data_call_error_e * err
<pre>int ML_DataCallStop (int32_t profile_idx, ml_data_call_error_e * err)</pre>	断开公网/私网 PDN 连接。 同步接口，下发建立 PDN 命令到 modem，modem 返回后，此接口才返回。 参数： 输入： int32_t profile_idx 输入： ml_data_call_ip_family_e ip_family 输出： ml_data_call_error_e * err
<pre>int ML_DataCallStart_Ext (int32_t profile_idx, ml_data_call_error_e * err)</pre>	建立公网/私网 PDN 连接。 异步接口，下发建立 PDN 命令后直接返回，不等 modem 执行结果。建立 PDN 的状态通过 ML_DataCallInit 注册的状态通过 ML_DataCallInit 注册的 callback 函数回调给 APP。 参数：

接口/结构体	描述
	输入： const ml_data_call_s * data_call 输出： ml_data_call_error_e * err
<pre>int ML_DataCallStop_Ext (int32_t profile_idx, ml_data_call_error_e * err)</pre>	断开公网/私网 PDN 连接。 异步接口，下发断开 PDN 命令后直接返回，不等 modem 执行结果。断开 PDN 的状态通过 ML_DataCallInit 注册的状态通过 ML_DataCallInit 注册的 callback 函数回调给 APP。 参数： 输入： char profile_idx 输入： ml_data_call_ip_family_e ip_family 输出： ml_data_call_error_e * err
<pre>int ML_getDataCallReason (int32_t profile_idx, ML_DataCallFailCause * reason)</pre>	PDN 建立失败时，获取 failcause。 参数： 输入： profile_idx 输出： reason
<pre>Int ML_DataCallInfoGet(char profile_idx, ml_data_call_ip_family_e ip_family, ml_data_call_info_s *info, ml_data_call_error_e *err)</pre>	根据 profile_idx, ip_family 获取 data call 信息。 参数： 输入： char profile_idx 输入： ml_data_call_ip_family_e ip_family 输出： ml_data_call_info_s *info 输出： ml_data_call_error_e *err
<pre>int ML_SetDataDefaultSim (int32_t slotId)</pre>	双卡情况下，切换数据主卡。 切换数据主卡前，比如从 slot0 切换到 slot1，需先断掉 slot0 上左右的 PDN 连接，调用 ML_SetDataDefaultSim(1)，再建立 slot1 上的 PDN 数据连接。 参数： 输入： int32_t slotId

接口/结构体	描述
int ML_GetDataDefaultSim ()	双卡情况下，获取数据主卡 slot。返回值是获取到的主卡 slot 值。

1.2.2.5 电话接口及调用时序说明

电话接口均需要基于网络已注册的情况下调用。

表 1-6. 电话接口说明

接口/结构体	描述
<pre>typedef struct { int32_t CallId; char PhoneNum[32]; E_ML_VCALL_STATE_TYPE_T State; int32_t call_end_reason; } ML_VCALL_INFO_T;</pre>	电话信息结构体。 CallId: 当前电话 index; PhoneNum[32]: 对端电话号码; State: 电话状态; call_end_reason: 电话挂断;
<pre>typedef void (*ML_VCALL_MSGCB_T) (ML_VCALL_INFO_T *pvsMsg);</pre>	电话回调函数。
<pre>Int32_t ML_VcallInit(ML_VCALL_MSGCB_T cb_func)</pre>	注册电话回调函数。 来电时触发回调函数，来电信息通过回调函数返回。 参数： 输出： ML_VCALL_MSGCB_T cb_func; 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
<pre>int32_t ML_VcallInitWithSlot(int slot_id, ML_VCALL_MSGCB_T cb_func)</pre>	按 slot 注册电话回调函数。 来电时触发回调函数，来电信息通过回调函数返回。 参数： 输出： ML_VCALL_MSGCB_T cb_func; 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
<pre>Int ML_VcallStart(const char *PhoneNumber)</pre>	拨出电话。 参数： 输入： const char *PhoneNumber; 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。

接口/结构体	描述
int32_t ML_VcallStartWithSlot(int32_t slotId, const char *PhoneNumber)	By Slot 拨出电话。
Int ML_VcallAnswer(void)	接听电话。 只有来电状态时才能接听电话，其他状态无法接听，会 return error。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallAnswerWithSlot(int32_t slotId);	By slot 接听电话。
Int ML_VcallEnd (void)	挂断电话。 只有通话状态时才能挂断电话，其他状态无法挂断，会 return error。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallEndWithSlot(int32_t slotId)	By slot 挂断电话。
int32_t ML_VcallCancel (void)	取消电话回调函数 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallCancelWithSlot(int32_t slotId)	By slot 取消电话回调函数
int32_t ML_VcallCliStart (const char * PhoneNumber, int32_t cli)	播出电话，并设置对端是否显示。 参数： 输入： const char *PhoneNumber; int32_t cli 0: 默认 1: 禁止显示 2: 允许显示 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallCliStartWithSlot(int32_t slotId, const char *PhoneNumber, int32_t cli)	By slot 播出电话，并设置对端是否显示
int32_t ML_VcallControl (int32_t controlType, int32_t call_id	挂起和恢复电话。 参数：

接口/结构体	描述
)	输入： int32_t controlType 0: 挂起电话 1: 恢复电话 int32_t call_id 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallControlWithSlot(int32_t slotId, int32_t controlType, int32_t call_id)	By Slot 挂起和恢复电话。
void ML_SetAutoAnswerMode (bool onoff)	设置自动接听。 参数： 输入： bool onoff
int32_t ML_VcallDtmfStart (const char * Number)	菜单导航拨号开始 参数： 输入： const char * Number 拨号号码 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallDtmfStartWithSlot(int32_t slotId, const char *Number)	By Slot 菜单导航拨号开始
int32_t ML_VcallDtmfStop (void)	菜单导航拨号结束 注意，此 API 需要搭配 VcallDtmfStart 使用，Start 之后按 Stop 结束 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t ML_VcallDtmfStopWithSlot(int32_t slotId)	By Slot 菜单导航拨号结束
int32_t ML_VcallGetCallWaiting (int32_t serviceClass, bool * enable, int32_t * serviceClassReturn)	查询呼叫等待状态。 参数： 1: 语音 (Voice) 2: 数据 (Data, 指所有承载服务) 8: 短消息服务 (SMS) 16: 数据电路同步 (Data Circuit Sync) 32: 数据电路异步 (Data Circuit Async)

接口/结构体	描述
	64: 专用分组接入 (Dedicated Packet Access) 128: 专用 PAD 接入 (Dedicated PAD Access) 输入: int32_t serviceClass 指定服务类别 输出: bool * enable int32_t * serviceClassReturn
int32_t ML_VcallGetCallWaitingWithSlot(int32_t slotId, int32_t serviceClass, bool *enable, int32_t *serviceClassReturn)	By slot 查询呼叫等待状态
int32_t ML_VcallSetCallWaiting (bool enable, int32_t serviceClass)	设置呼叫等待状态。 参数: 输入: int32_t serviceClass 指定服务类别 bool * enable
int32_t ML_VcallSetCallWaitingWithSlot(int32_t slotId, bool enable, int32_t serviceClass);	By Slot 设置呼叫等待状态。
int32_t ML_VcallStopCallId (int32_t call_id)	挂断指定 id 电话。 参数: 输入: int32_t call_id
int32_t ML_VcallStopCallIdWithSlot(int32_t slotId, int32_t call_id)	By slot 挂断指定 id 电话
int32_t ML_Vcall_Get_StatusWithSlot(int32_t slotId, E_ML_VCALL_STATE_TYPE_T *pveState)	获取 slot 当前电话状态 参数: 输入: slotId 输出: E_ML_VCALL_STATE_TYPE_T *pveState
int32_t ML_VcallHoldWithSlot(int32_t slotId)	Hold 对应 Slot 的电话。 参数: 输入: int32_t slotId

1.2.2.6 短信接口及调用时序说明

短信接口均需要基于网络已注册的情况下调用。

表 1-7. 短信接口说明

接口/结构体	描述
<pre>typedef enum { E_ML_SMS_FORMAT_GSM_7BIT = 0, E_ML_SMS_FORMAT_BINARY_DATA = 1, E_ML_SMS_FORMAT_UCS2 = 2, E_ML_SMS_FORMAT_IRA = 3, }E_ML_SMS_FORMAT_T;</pre>	短信格式枚举定义
<pre>typedef struct { E_ML_SMS_FORMAT_T format; char PhoneNum[ML_SMS_MAX_ADDR_LENGTH]; int32_t SmsDataLen; char SmsData[ML_SMS_MAX_MT_MSG_LENGTH]; } ML_SMS_INFO_T;</pre>	短信信息结构体
<pre>typedef void (*ML_SMS_RXMSGCB_T) (ML_SMS_INFO_T *pvsMsg);</pre>	短信回调函数
<pre>Int ML_SmsInit(ML_SMS_RXMSGCB_T cb_func)</pre>	注册短信回调函数 接收短信时触发回调函数，短信信息通过回调函数返回。 参数： 输出： ML_SMS_RXMSGCB_T cb_func;
<pre>Int32_t ML_Sms_Sent(const ML_SMS_INFO_T *pvsSms)</pre>	发送短信。 异步接口，发送短信请求发送后会异步等待 modem 返回响应信息，不会卡住线程。 参数： 输入： const ML_SMS_INFO_T *pvsSms;
<pre>int32_t ML_Sms_SentWithSlot(int32_t slotId, const ML_SMS_INFO_T *pvsSms)</pre>	By Slot 发送短信。
<pre>int32_t ML_SmsInitWithSlot (int32_t slot_id, ML_SMS_RXMSGCB_T cb_func)</pre>	初始化短信模块，并注册槽位 id 和回调函数。 函数返回值为 int 类型，成功则返回 0，失败则返回-1。

接口/结构体	描述
<pre>int32_t ML_SmsInitNoDecoder (int32_t slot_id, ML_SMS_IND_T cb_func)</pre>	收到新短信时显示，默认关闭，若要开启此功能 请打开：meta/meta-mediatek- mtxxxxx/conf/machine/autoxxxx.conf 然后修改 MTK_SMS_CODECSUPPORT = "no" => MTK_SMS_CODECSUPPORT = "yes"。 函数返回值为 int 类型，成功则返回 0；错误则返回 -1。
<pre>int sendSmsNoEncoderAsync (int slot_id, char * smscPdu, char * pdu)</pre>	发送短信。根据返回的错误，如果发送短信失 败，呼叫者决定重新发送。 RadioError:SMS_SEND_FAIL_RETRY 表示重试（即错 误原因是 332）而 RadioError:GENERIC_FAILURE 表 示不重试（即错误原因是 500）默认为禁用。启用 此功能，请打开：meta/meta-mediatek- mtxxxxx/conf/machine/autoxxxx.conf 然后修改 MTK_SMS_CODECSUPPORT = "no" => MTK_SMS_CODECSUPPORT = "yes"。 函数返回值为 int 类型。成功时返回 0；错误时返回 -1。
<pre>int ML_sendSmsNoEncoderResponseCb (int32_t slot_id, ml_send_sms_no_encoder_async_response response)</pre>	sendSmsNoEncoderAsync 响应函数

1.2.2.7 IMS 接口及调用时序说明

表 1-8. IMS 接口说明

接口/结构体	描述
<pre>Int ML_EnableIms(uint8_t on_off)</pre>	开启/关闭 IMS 功能 传入参数：on_off 为 1 表示开启 IMS 功能。 On_off 为 0，表示关闭 IMS 功能。
<pre>Int ML_GetImsRegState(int8_t *reg_state)</pre>	获取 IMS 注册状态 传出参数 reg_state 值为 1，表示 IMS 注册上，reg_state 值为 0 则 表示 IMS 没有注册上。

接口/结构体	描述
int ML_EnableVolte (uint8_t on_off)	为 volte 启用 ims。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int ML_EnableVonn (uint8_t on_off)	为 vonn 启用 ims，如果需要禁用 volte 和 vonn，请使用 ML_EnableIms(0)。 此 API 仅支持 slot0。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t setVoNrEnabled (int32_t slot_id, bool enable)	设置语音降噪启用状态。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。
int32_t isVoNrEnabled (int32_t slot_id, bool * enable)	查询当前 Voice NR 启用状态。 函数返回值为 int 类型。成功时返回 0; 错误时返回 -1。

1.2.2.8 Ecall 接口及调用时序说明

表 1-9. Ecall control 接口说明

接口/结构体	描述
typedef struct { int32_t call_id; uint32_t length; unsigned char msd_data[ML_ECALL_MSd_MAX_LENGTH]; }ml_ecall_set_msds;	设置 MSD 时传入结构体参数定义。
typedef struct { int32_t arg_num; int32_t type; char address[128]; }ml_ecall_set_num;	设置 test number 及 reconfig number 时传入结构体参数定义。
typedef enum { ML_EMER_CAT_MANUAL_ECAll = 1, ML_EMER_CAT_AUTO_ECAll = 2, }ml_ecall_category;	发生 Ecall 类别的结构体。
typedef enum { ML_ECAll_TEST = 1, ML_ECAll_EMERGENCY = 2, ML_ECAll_RECONFIG = 3, }ml_ecall_variant;	Ecall 类型的结构体。

接口/结构体	描述
<pre>typedef enum{ ML_DOMAIN_AUTO = 0, /* Automatic mode - LTE(IMS), WG(CS), 1x(C2K) */ ML_DOMAIN_CS_ONLY = 1, /* CS domain only - WG(CS) */ ML_DOMAIN_3GPP_ONLY = 2, /* 3GPP only - LTE(IMS), WG(CS) */ ML_DOMAIN_3GPP2 = 3, /* 3GPP2 only - 1x(C2K)) */ ML_DOMAIN_IMS_1xCS = 4, /* IMS and 1x CS only - LTE(IMS), 1x(C2K) */ ML_DOMAIN_CS_1x = 5, /* WG CS and 1x CS only - WG(CS), 1x(C2K) */ ML_DOMAIN_IMS_ONLY = 6, /* only IMS call allowed */ }ml_ecall_domain;</pre>	Ecall 通话的网络制式结构体。
<pre>typedef struct{ ml_ecall_category ecall_cat; ml_ecall_variant ecall_variant; char address[20]; uint32_t length; unsigned char msd_data[ML_ECALL_MSD_MAX_LENGTH]; ml_ecall_domain domain; }ml_ecall_req_msg;</pre>	请求拨出 Ecall 时传入结构体参数定义。
<pre>typedef struct{ int32_t data1; int32_t data2; int32_t data3; int32_t data4; }ml_ecall_pri;</pre>	Ecall 优先级参数结构体。data1>data2>data3>data4。传入参数应为 1 2 3 4，含义分别为： 1：客户设定的 Ecall URI 2：USIM 保存的 Ecall URI 3：客户设定的 ECall 号码 4：USIM 保存的 Ecall 号码
<pre>typedef enum{ E_ML_ECALL_SENDING_START = 1, E_ML_ECALL_SENDING_MSD = 2, E_ML_ECALL_LACK_RECEIVED = 3, E_ML_ECALL_ALACK_POSITIVE_RECEIVED = 4, E_ML_ECALL_ALACK_CLEARDOWN_RECEIVED = 5, E_ML_ECALL_DIALING = 9, E_ML_ECALL_ALERTING = 10,</pre>	Ecall 上报状态的消息类型枚举定义。

接口/结构体	描述
<pre> E_ML_ECALL_ACTIVE = 11, E_ML_ECALL_DISCONNECTED = 12, E_ML_ECALL_IMS_ACTIVE = 13, E_ML_ECALL_IMS_DISCONNECTED = 14, E_ML_ECALL_ABNORMAL_HANGUP=15, E_ML_ECALL_IMS_MSD_ACK = 20, E_ML_ECALL_IMS_UPDATE_MSD = 21, E_ML_ECALL_IMS_IN_BAND_TRANSFER = 22, E_ML_ECALL_IMS_MSD_NACK = 23, E_ML_ECALL_IMS_SRVCC = 24, E_ML_ECALL_ONLY_DEREGISTRATION = 31, E_ML_ECALL_MAY_DEREGISTER = 32, E_ML_ECALL_PSAP_CALLBACK_START = 40, E_ML_ECALL_PSAP_CALLBACK_IMS_UPDATE_MSD = 41, E_ML_ECALL_T2_TIMEOUT = 52, E_ML_ECALL_T5_TIMEOUT = 55, E_ML_ECALL_T6_TIMEOUT = 56, E_ML_ECALL_T7_TIMEOUT = 57, E_ML_ECALL_UNSPECIFIED = 0xffff, }ML_ECall_Indication; </pre>	
<pre> typedef struct{ int32_t purpose; int32_t mode; int32_t timer1; int32_t timer2; }ml_ecall_time; </pre>	purpose 表示用途，0 表示紧急呼叫（目前仅支持 0，未来可能会扩展） mode 表示模式，1 表示设置定时器；0 表示重置定时器 Timer1 表示定时器 1，用于设置紧急呼叫的定时器值（分钟） Timer2 表示定时器 2，用于设置恢复/重新配置呼叫的定时器值（分钟）
<pre> typedef enum { ECALL_NONE = 0, ECALL_ONLY = 1, ECALL_AND_NORMAL = 2, }ml_ecall_oprt_mode; </pre>	Ecall 工作模式
<pre> typedef enum { ECALL_TYPE_UNKNOWNN = 0, EN16454_ECALL = 1, GOST_ECALL = 2, NG_ECALL = 3, }ml_ecall_type; </pre>	Ecall 类型

接口/结构体	描述
typedef struct{ ML_ECall_Indication ind; int call_id; } ML_ECALL_IND_T;	Ecall 上报状态的消息结构体。
typedef void (*ML_ECALL_MSGCB_T)(ML_ECALL_IND_T *pvMsg);	Ecall 上报状态消息的回调函数。
int32_t ML_EcallIndicationInit(ML_ECALL_MSGCB_T cb_func);	注册 Ecall 状态变化 callback 函数，当 Ecall 状态发生变化时回调 cb_func 函数。 参数： 输入: ML_ECALL_MSGCB_T cb_func
int32_t ML_ResetIvs(void);	重置 modem 的 ecall 状态. 重连 audio。
int32_t ML_SetMSD(ml_ecall_set_msd* msd);	设置 ecall 的 msd 数据。 参数： 输入: ml_ecall_set_msd* msd
int32_t ML_SetTestNumber(ml_ecall_set_num* test_num);	设置 test number 或者 URI。 参数： 输入: ml_ecall_set_num* test_num
int32_t ML_SetReconfNumber(ml_ecall_set_num* reconf_num);	设置 reconfig number 或者 URI。 参数： 输入: ml_ecall_set_num* test_num
int32_t ML_MakeFastEcall(ml_ecall_req_msg* msg);	拨出 ecall 同时传入 msd 数据。 参数： 输入: ml_ecall_req_msg* msg
int32_t ML_SetEmsdpri(ml_ecall_pri* pri);	设置 ecall 优先级类别。默认优先级是“1>3>2>4”。 参数： 输入: ml_ecall_pri* pri
int32_t ML_SetNadDeregTime(ml_ecall_time* time);	设置 NAD 注销定时器 参数： 输入: ml_ecall_time* time
int32_t ML_SetNadRegState(uint8_t state);	设置 NAD 注册状态

接口/结构体	描述
	参数: 输入: uint8_t state
int32_t ML_SetOprtMode(ml_ecall_oprt_mode mode);	设置 ecall 工作模式 参数: 输入: ml_ecall_oprt_mode mode
int32_t ML_GetOprtMode(ml_ecall_oprt_mode *mode);	获取 ecall 工作模式 参数: 输入: ml_ecall_oprt_mode *mode
int32_t ML_EcallPrecondition(void);	设置 ecall 前提条件, 主要是降低功耗的措施, 认证阶段无需调用
int32_t ML_SetEcallType(ml_ecall_type type);	设置 ecall 类型 参数: 输入: ml_ecall_type type
int32_t ML_GetEcallType(ml_ecall_type *type);	获取 ecall 类型 参数: 输入: ml_ecall_type *type
int32_t ML_SetGostAttempts(int32_t attempts);	设置 GostEcall 的尝试次数 参数: 输入: int32_t attempts
int32_t ML_SetGostInterval(int32_t interval);	设置 GostEcall 的间隔时间 参数: 输入: int32_t interval
int32_t ML_SetGostDefault();	设置 GostEcall 的默认参数

1.2.2.9 休眠唤醒 Modem 相关接口及调用时序说明

表 1-10. AT 黑名单接口说明

接口/结构体	描述
Int ML_SetUnsolResponseFilter(ML_UnsolResponseFilter filter)	进入 Suspend 前，开启 AT URC 过滤功能： 1. 调用 ML_SetUnsolResponseFilter(0)
int ML_SetUnsolResponseFilterWithSlot(int32_t slotId, ML_UnsolResponseFilter filter);	退出 Suspend 后，关闭 AT URC 过滤功能： 1. 调用 ML_SetUnsolResponseFilter(0xFF)

1.2.2.10 发送接受 Modem AT Command 相关 API

表 1-11. AT command 接口说明

接口/结构体	描述
int32_t ML_SendAT(const char* atCmd, char* finalRsp, uint32_t resp_len, int64_t timeout_ms); int32_t ML_SendATWithSlot(int slotId, const char* atCmd, char* finalRsp, uint32_t resp_len, int64_t timeout_ms);	发送 AT 命令接口，出于安全考虑，目前仅支持少数几种。 如果您想添加此列表，请修改 libmtkrilutils.c 文件中的 sAtCmdForOemHook 结构体。 ML_SendAT 将使用默认槽位 ID 调用，该槽位 ID 来自 ML_GetDataDefaultSim()。 位置： src/telephonyware/3.1/hardware/ril/platformlib/common/libmtkrilutils/libmtkrilutils.c
typedef void (*ML_OEM_HOOK_CB_T)(int 32_t slot_id, const char* urc, int32_t urc_length); int32_t ML_OmeHookUrcWithSlot(int 32_t slot_id, ML_OEM_HOOK_CB_T cb_func)	用于从调制解调器向框架发送未经请求的 AT 命令，以进行内部测试。

表 1-12. MIPC keep alive 接口说明

接口/结构体	描述
<pre>typedef struct { ML_keepaliveType type; char sourceAddress[MAX_INADDR_LEN]; int32_t sourcePort; char desinationAddress[MAX_INADDR_LEN]; int32_t desinationPort; int32_t maxKeepAliveIntervalMillis; int32_t profile_idx; } ML_keepaliveRequest typedef struct { KEEPALIVE_ACTIVE = 0, KEEPALIVE_INACTIVE, KEEPALIVE_PENDING } ML_keepaliveStatusCode typedef struct { int32_t sessionHandle; ML_KeepaliveStatusCode code; } ML_keepaliveStatusCode int ML_StartKeepAlive (ML_KeepaliveRequest req, ML_KeepaliveStatus * ml_status) int ML_StopKeepAlive (uint32_t sessionHandle) int ML_SetKeepAliveStatusInd (ml_keepalive_cb_t evt_cb)</pre>	通过发送数据包来保持互联网 PDN/PDU 会话处于连接模式，而不唤醒 AP 处理器。 流程： 1. AP 调用 ML_StartKeepalive。回调 keepaliveStatus 状态为 pending，表示等待 AP 发送第一个 UL packet。 2. 等 modem 收到 AP 发送的第一个 UL packet 后，回调 keepaliveStatus 状态为 active，表示 MD 能发送 keep-alive 封包了。 3. 若出现异常，当 modem 最终收不到网络 keep-alive 响应信息后，回调 keepaliveStatus 为 inactive。

1.3 常见问题/故障排除

1.3.1 SIM/CALL/SMS/Telephony 驻网相关

1. 如何关闭/开启指定网络类型，参考[FAQ17447] [NW]关闭指定网络类型。
2. SIM 测项失败 预先调试：
 - (1) IMEI 初始值是空的，需要先烧录才能获取到。
 - (2) 若 SIM 识别异常，抓取包含开机过程的测试场景 mtklog 协助进一步分析。
3. 驻网失败 预先调试：
 - (1) 确认平台有烧录 RF/IMEI，正确安装天线，SIM 卡正确插入且不欠费。
 - (2) 若 (1) 确认后仍然无法驻网，抓取包含开机过程的测试场景 mtklog 协助进一步分析。
4. CALL/SMS 失败 预先调试：

(1) 使用以下命令确认平台是否有成功驻上网，若未驻网成功，先解决驻网问题。

```
adb shell /usr/bin/mlclient_test --gtest_filter=MLClientTest.ML_GetNetState
```

(2) 若 (1) 步骤已经驻网成功，抓取测试场景 mtklog 协助进一步分析。

1.3.2 Data 相关

1. APN 配置文件的修改，[FAQ21414] MTK apns-conf.xml configuration guide。

T-Box 使用的配置文件命名为 apns-conf.xml；代码路径：`src/telephonyware/3.1/libvendor-ril/apn/resource/apns-conf.xml`；平台中的安装路径：`/system/etc/tele/apns-conf.xml`

若直接修改 `/system/etc/tele/apns-conf.xml`，需删除 `apn.db`，重启生效。

`apn.db` 路径：`/data/vendor/telephony/apn.db`

1.3.3 网络相关

网络建立成功的检查：

1. 网络接口 up 并且 IP 地址存在，透过 `ifconfig` 命令确认。
2. 网络 IP rule & IP route & DNS 设置，透过 `ip rule/ip route/dumpsys netd/dumpsys dnsresolver` 命令确认。

网络连通性的检查：

手动 ping 某 IP 或某网址是否能 ping 通。

1.3.4 VOLTE 和 VONR 相关

如何默认开机自动开启或者关闭 VOLTE and VONR？

1. 当前系统默认开机自动开启 VOLTE and VONR 功能(前提是 SIM 本身开通了相关功能)。
2. 可通过改变 YOCTO 文件 `meta/meta-mediatek-mt8668/conf/machine/auto8668p1_64.conf` 里面 `MTK_VOLTE_SUPPORT` or `MTK_VONR_SUPPORT` 为 yes 或者 no，来改变是否支持开机启动。
3. 也可以通过 mlclient 相关 API 也改变 VONR 和 VOLTE 的状态

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。